Project Report: Group 42

BookMyShow Information System

1. Problem Description and Assumptions

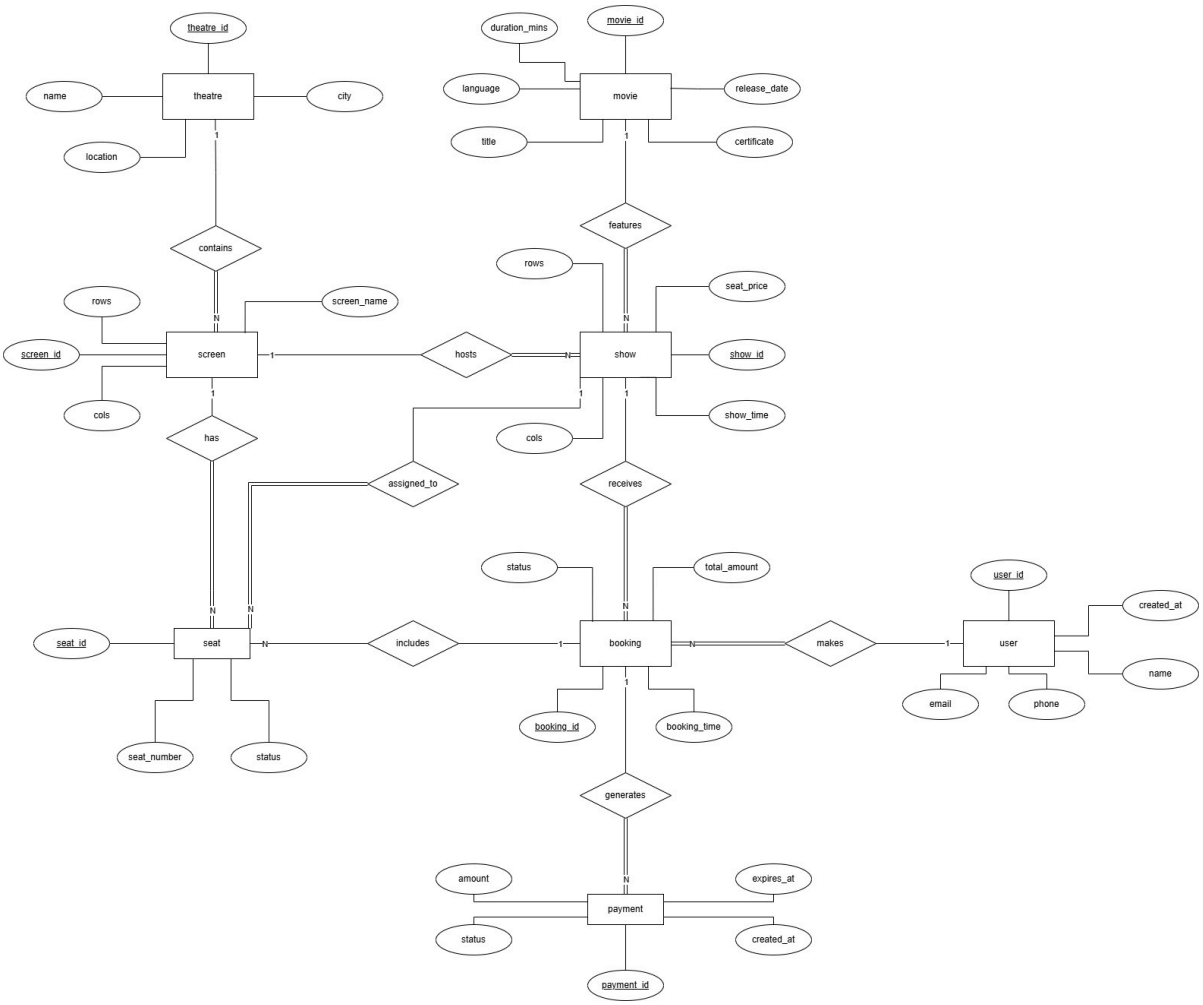
Problem Description

The objective of this project is to design and implement a complete database-backed information system for a movie ticket booking platform. The system manages users, movie listings, theater locations, and show schedules, allowing for seamless seat selection and payment processing.

Assumptions

- Each movie show is scheduled for a 3 hour block to prevent overlapping on the same screen.
- Seats are show-specific; a new set of seats is initialized for every individual showtime to track availability accurately.
- A booking remains in a "Pending" status until a successful payment signal is received from the simulation module.

2. Conceptual Design (ER Modeling)



3. Table-Wise Constraints

1. User Table

- **Primary Key**: user_id uniquely identifies each registered user.
- **Unique Constraints**: Both email and phone are marked as UNIQUE to prevent multiple accounts from using the same contact details.
- **Entity Integrity**: name, email, and phone are defined as NOT NULL to ensure all profiles have essential identification data.

2. Movie Table

- **Primary Key**: movie_id uniquely identifies each film entry.
- **Check Constraints**:
  - chk_movie_duration: Validates that duration_mins is greater than 0.
  - chk_movie_release: Ensures the release_date is not set in the future relative to the current date.

3. Theatre Table

- **Primary Key**: theatre_id uniquely identifies each theater location.
- **Key Constraints**: A composite UNIQUE constraint is applied to the combination of name and city to prevent duplicate theater branches in the same city.

4. Screen Table

- **Primary Key**: screen_id uniquely identifies each screen.
- **Referential Integrity**: theatre_id is a foreign key referencing the Theatre table with ON DELETE CASCADE.
- **Key Constraints**: The pair of theatre_id and screen_name is UNIQUE to ensure screen names are distinct within a single theater.

5. Show Table

- **Primary Key**: show_id uniquely identifies a specific movie screening.
- **Referential Integrity**: References movie_id and screen_id with ON DELETE CASCADE.
- **Exclusion Constraint**: excl_show_screen_time prevents a screen from hosting multiple shows simultaneously by checking for overlapping time ranges.
- **Check Constraint**: chk_show_price ensures the seat_price is always greater than 0.

6. Booking Table

- **Primary Key**: booking_id uniquely identifies each booking transaction.
- **Domain Constraints**: The status column is restricted to valid values defined in booking_status_enum ('Pending', 'Booked', 'Cancelled').
- **Check Constraint**: chk_booking_total ensures the total_amount is never negative.

7. Seat Table

- **Primary Key**: seat_id uniquely identifies each physical seat for a show.
- **Key Constraints**: show_id and seat_number must be UNIQUE together to prevent a seat from being double-booked for the same showtime.
- **Referential Integrity**: The booking_id is a foreign key that is set to NULL on delete, allowing seats to be released if a booking is removed.

8. Payment Table

- **Primary Key**: payment_id uniquely identifies each payment record.
- **Domain Constraints**: The status must match payment_status_enum values ('Pending', 'Success', 'Failed').
- **Referential Integrity**: References booking_id with ON DELETE CASCADE to maintain a direct link between a booking and its financial record.

<u>Database Implementation</u>

- **Relational Schema**: The system is implemented through SQL scripts creating 8 core tables: user, movie, theatre, screen, show, booking, payment, and seat.

- **Sample Data**: The database is prepopulated with representative movies and theaters across multiple cities to demonstrate functionality.

4. Front-End Interface Description

- **Technology**: Web-based interface built using React and Vite.
- **Functionalities**:
    - **Data Retrieval**: Users can search for movies and browse theater schedules by city.
    - **Data Insertion/Modification**: Provides dedicated components for user registration and creating new bookings.
    - **Admin Module**: Includes an administrative interface for managing movie listings, theaters, and screen layouts.